

Le modèle conceptuel de données (MCD)

Vite & Gourmand est une application de commande de menus traiteur à domicile. Dès le départ, j'ai fait face à l'enjeu de trouver comment organiser les données pour que tout fonctionne correctement : les clients, les menus, les plats, les commandes, les allergènes, etc. La première chose à prendre en compte était d'éviter que la BDD soit ingérable et pleine de doublons.

Dans les sites de vente en ligne, il est fréquent d'avoir une séparation claire entre ce qu'on vend (les menus), qui achète (les utilisateurs), et les transactions (les commandes). C'est exactement la logique que j'ai suivie.

Un menu n'appartient pas à une commande, il peut apparaître dans des dizaines de commandes différentes. De la même façon, un utilisateur peut passer plusieurs commandes en même temps. Ces relations "un vers plusieurs" m'ont naturellement conduit à séparer les tables.

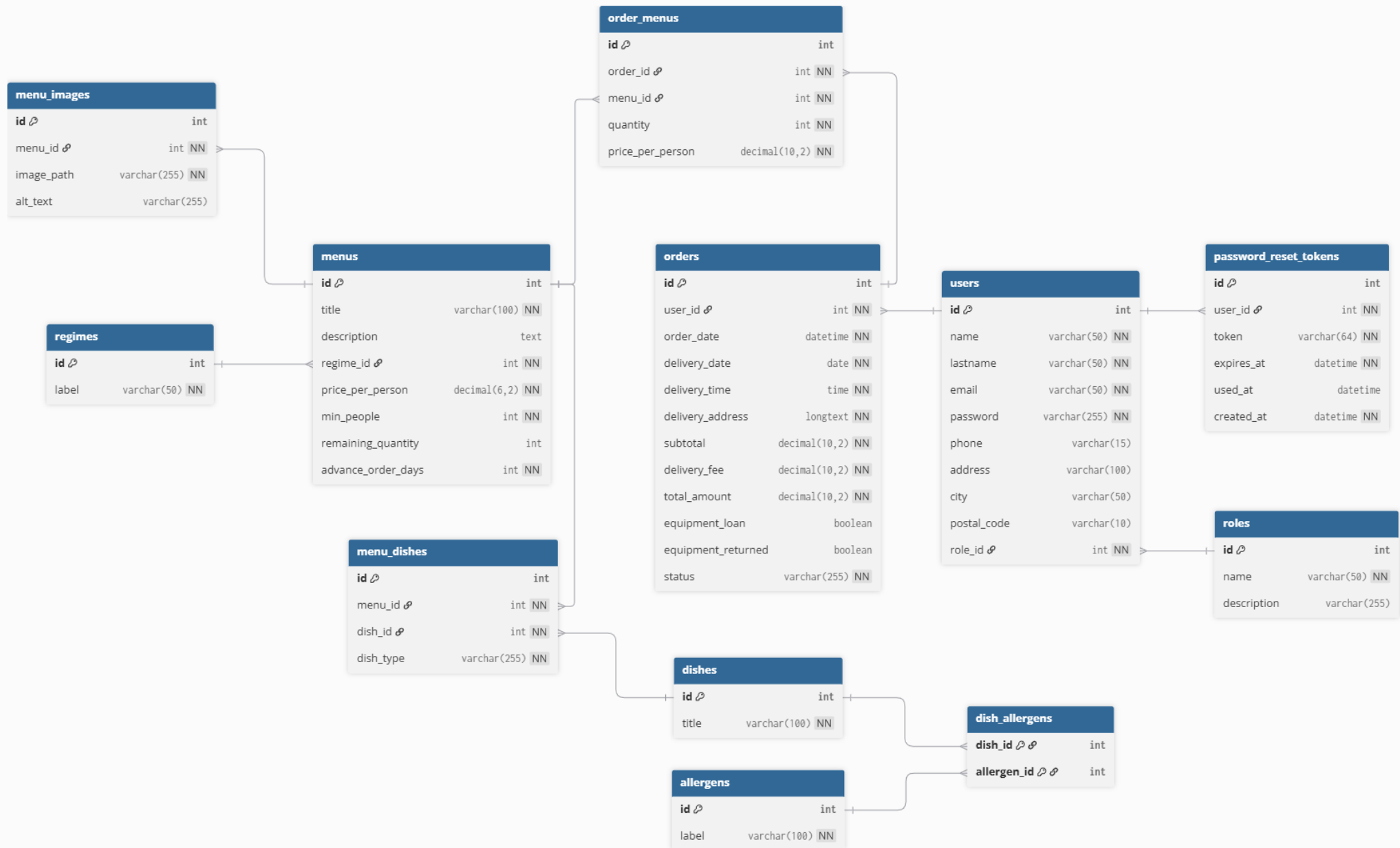
J'ai pris la décision de créer des tables de liaison dans les cas où la relation entre deux entités est de type "**plusieurs vers plusieurs**". Par exemple, *un menu est composé de plusieurs plats, et un même plat peut être dans plusieurs menus*.

J'ai donc créé la table menu_dishes pour faire le lien entre les deux, la table menus et la table dishes. De la même manière pour les allergènes : *un plat peut avoir plusieurs allergènes, et un allergène peut concerner plusieurs plats*, d'où la table dish_allergens.

Dans ces tables de liaison, j'ai parfois ajouté des informations supplémentaires propres à la relation elle-même : par exemple dans menu_dishes, j'ai ajouté une colonne dish_type pour préciser si le plat est une entrée, un plat principal ou un dessert, une information qui n'appartient ni au plat ni au menu seuls, mais à leur association. De la même façon, dans order_menus, j'ai stocké le prix par personne au moment de la commande, pour que le montant ne change pas si le prix du menu est modifié plus tard.

Définir une association cohérente et fonctionnelle a peut-être été l'une de choses la plus compliquées à développer. Pour y arriver, j'ai cherché quelques exemples de code sur GitHub, dans des projets d'applications de delivery.

Enfin, certaines tables ont été ajoutées pendant le développement, quand j'ai réalisé qu'il me manquait quelque chose. C'est le cas de password_reset_tokens, créée quand j'ai implémenté la fonctionnalité de réinitialisation de mot de passe, et de menu_images, ajoutée quand j'ai voulu associer plusieurs photos à un même menu. Jusqu'alors, je stockais les photos d'une façon qui ne me semblait pas optimale.

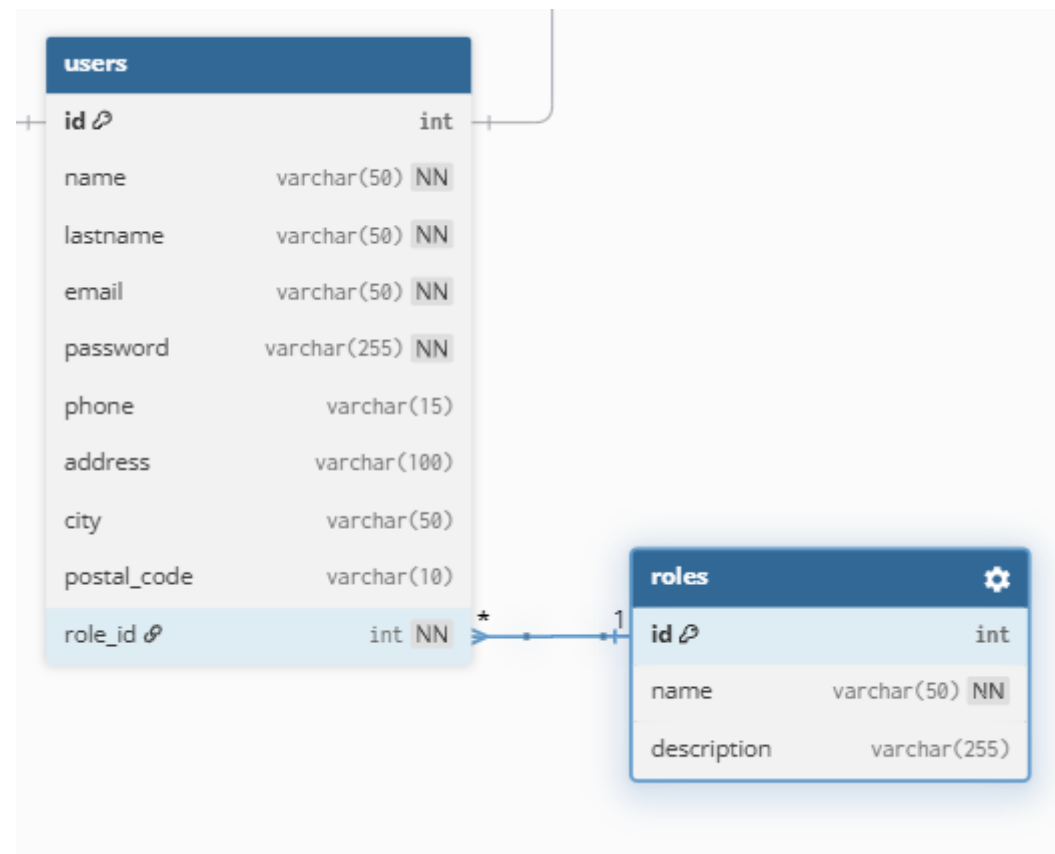


Analyse des relations entre les tables

roles → users = One-to-Many

Un rôle peut être attribué à plusieurs utilisateurs, mais un utilisateur n'a qu'un seul rôle à la fois.

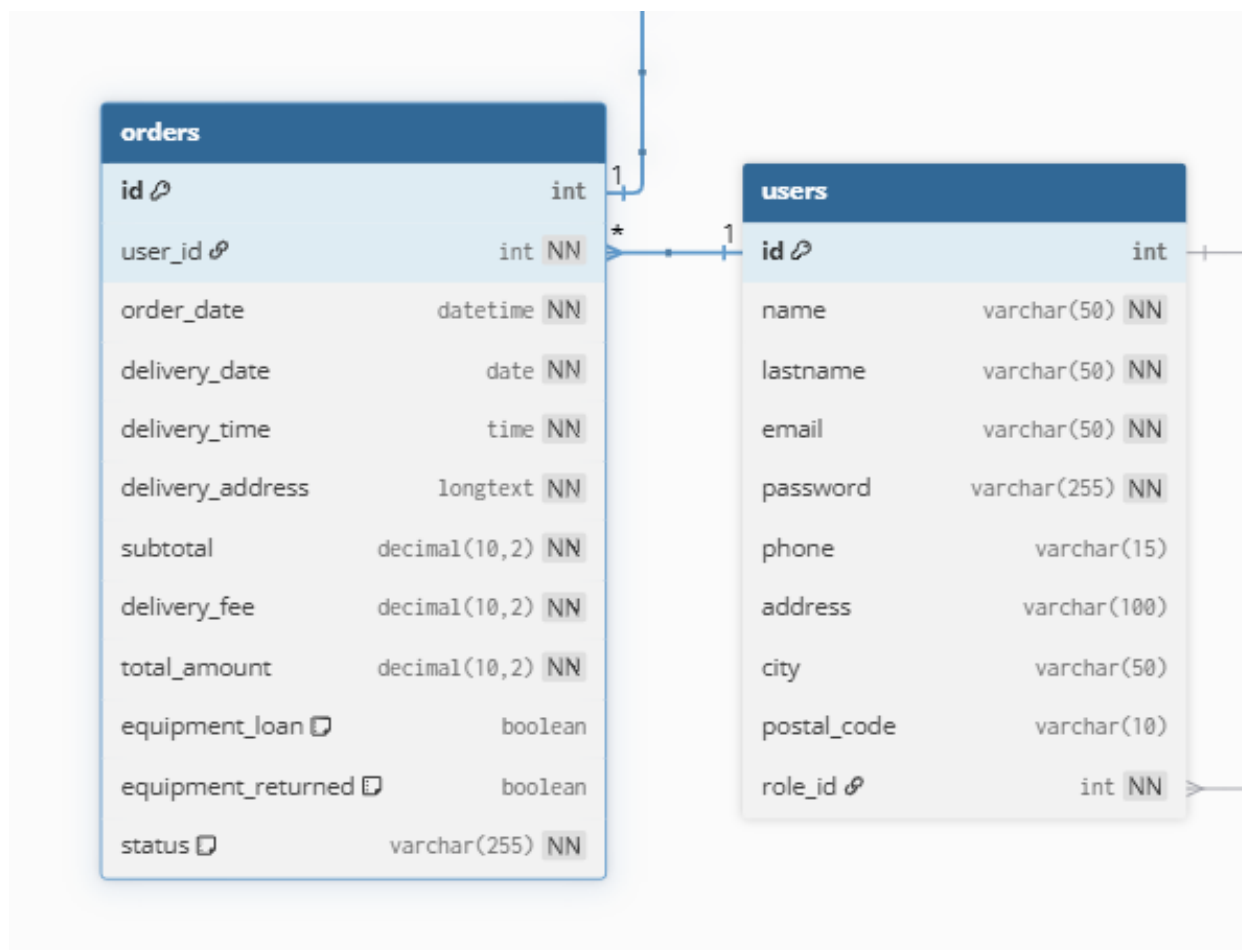
C'est la base du système d'identification : admin, staff_member ou client.



users → orders = One-to-Many

Un utilisateur peut passer plusieurs commandes au cours du temps, mais chaque commande appartient à un seul utilisateur.

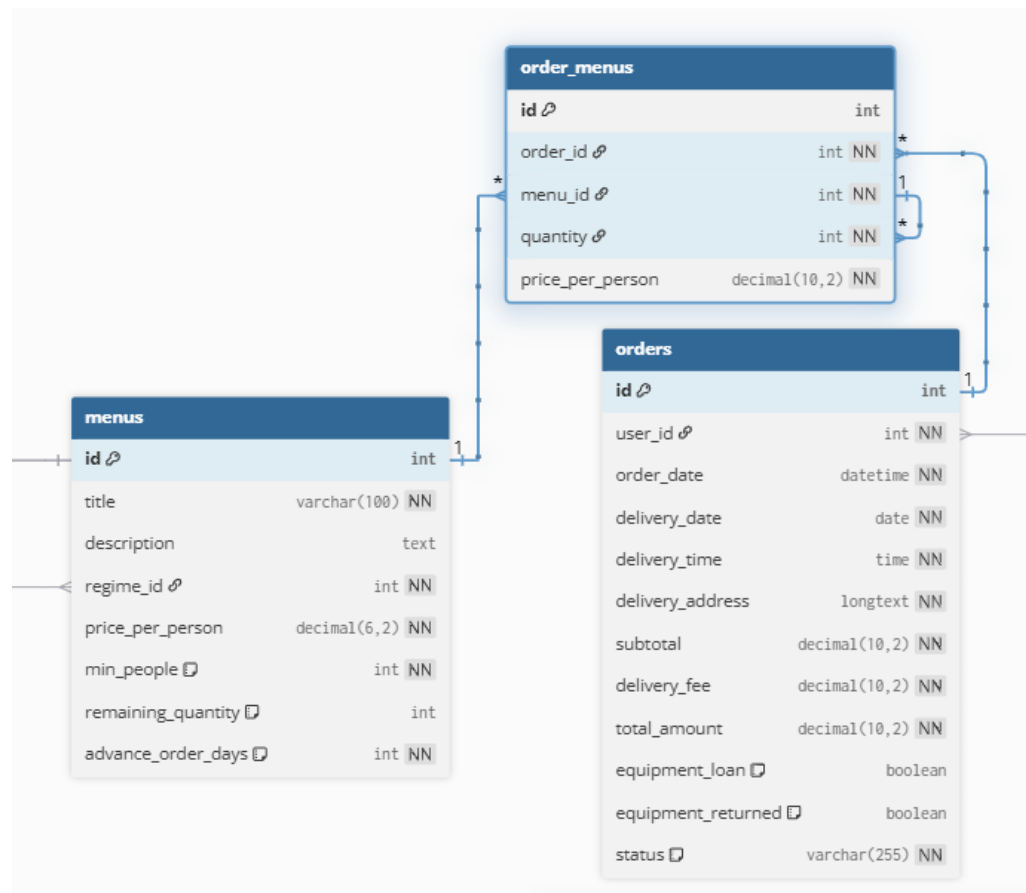
C'est la relation centrale de l'application côté client.



orders ↔ menus via order_menus = Many-to-Many

Une commande peut contenir plusieurs menus, et un menu peut être commandé dans plusieurs commandes différentes.

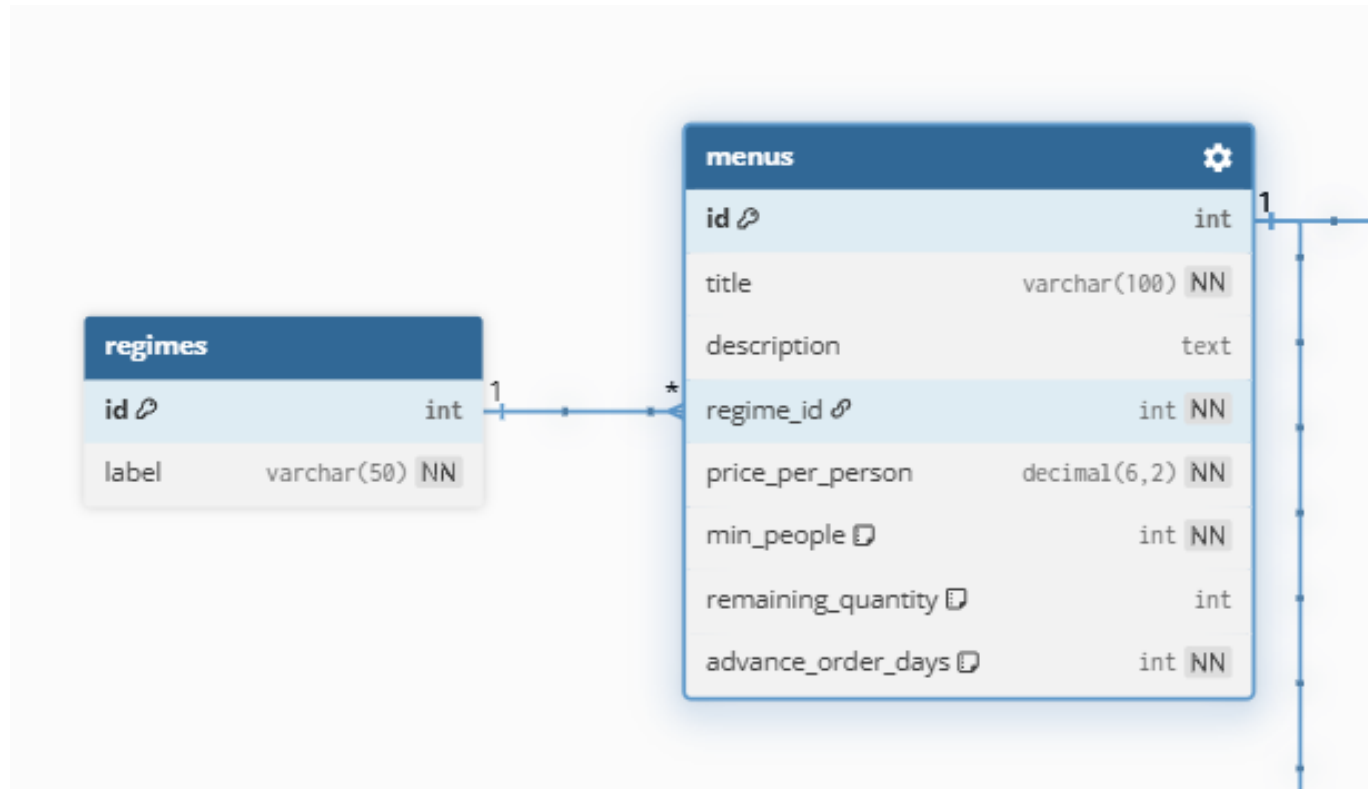
La table order_menus est une table de liaison enrichie avec quantity (nombre de personnes) et price_per_person (prix fixé au moment de la commande, indépendant du prix actuel individuel du menu).



regimes → menus = One-to-Many

Un régime alimentaire (Classique, Végétarien, Vegan, etc.) peut être associé à plusieurs menus, mais chaque menu a un seul régime.

Cela permet de filtrer les menus par type de régime.



menus ↔ dishes via menu_dishes = Many-to-Many

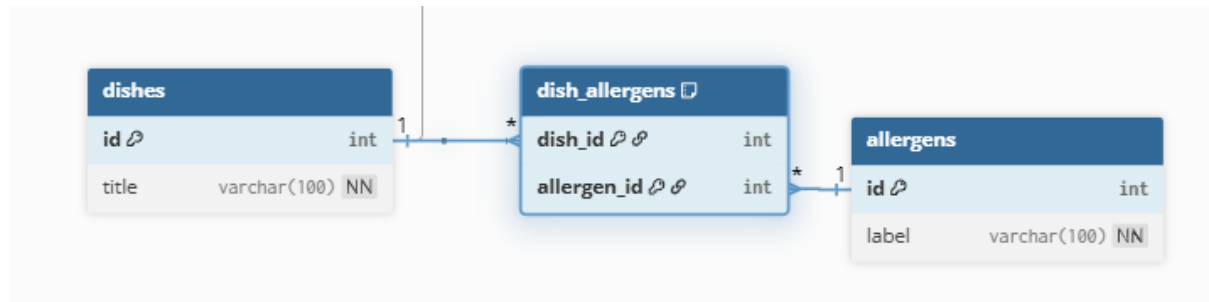
Un menu est composé de plusieurs plats (entrée, plat principal, dessert), et un même plat peut apparaître dans plusieurs menus.

La table menu_dishes enrichit la relation avec la colonne dish_type qui précise le rôle du plat dans le menu. Elle a aussi sa propre colonne id auto-incrémentée.



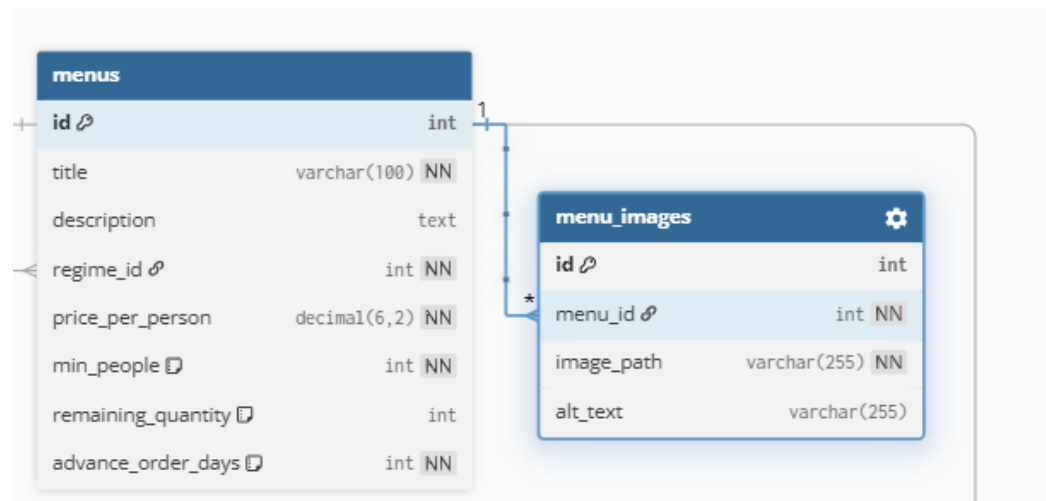
dishes ↔ allergens via dish_allergens = Many-to-Many

Un plat peut contenir plusieurs allergènes, et un même allergène peut être présent dans plusieurs plats. La table dish_allergens est une table de liaison pure (clé primaire) sans données supplémentaires.



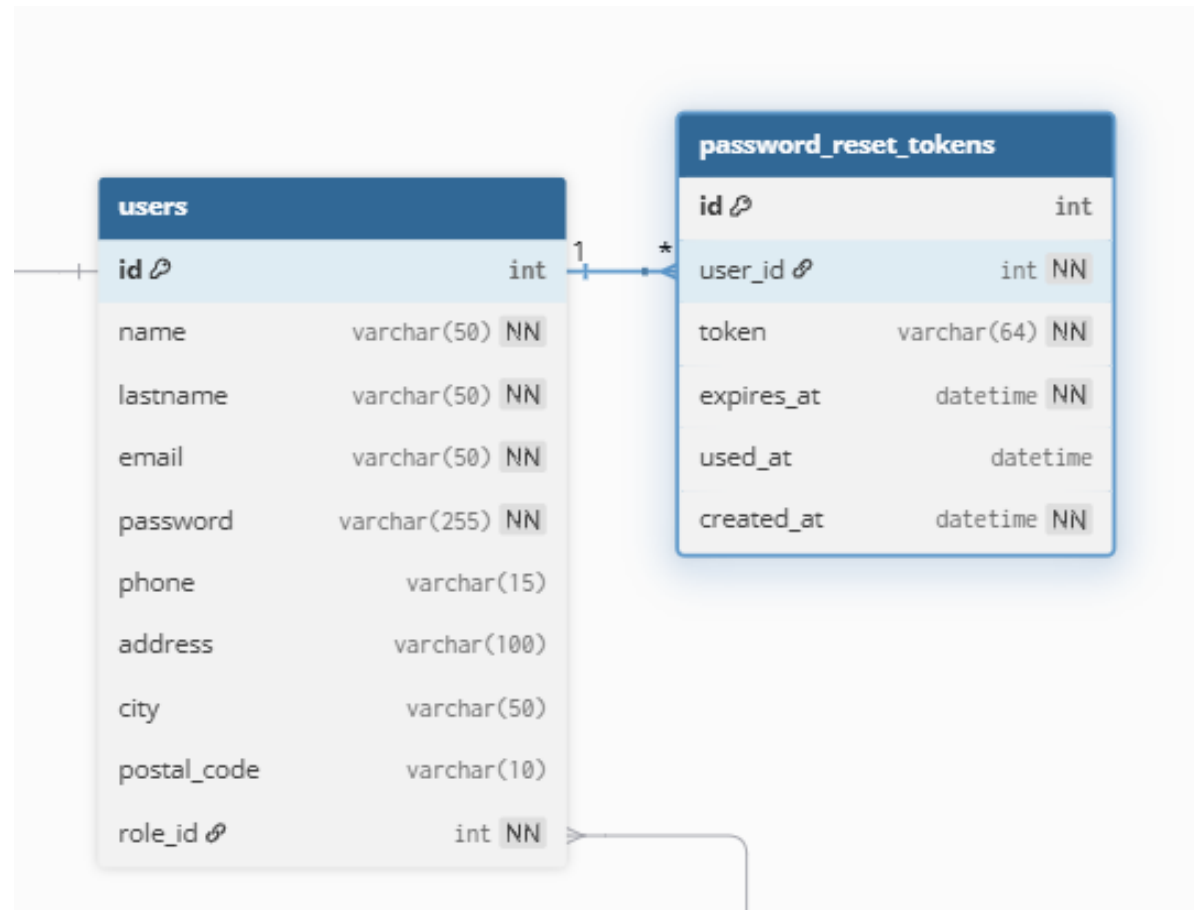
menus → menu_images = One-to-Many

Un menu peut avoir plusieurs images (galerie photos), mais chaque image appartient à un seul menu. La suppression d'un menu déclenche la suppression en cascade de ses images.



users → password_reset_tokens = One-to-Many

Un utilisateur peut avoir plusieurs tokens de réinitialisation dans l'historique (un par demande), mais chaque token appartient à un seul utilisateur.



Diagrammes de cas d'utilisation et diagrammes de séquence

Les diagrammes sont un outil qui nous permet de visualiser comment l'application va fonctionner dans son ensemble. C'est une étape qui est essentielle : sans une vue claire de qui fait quoi et de ce qui se passe à chaque étape, il est très facile de se perdre ou d'oublier des cas à gérer. Il y a deux types de diagrammes qui sont des outils classiques dans la conception d'un logiciel.

Le premier outil est le **diagramme de cas d'utilisation**. Il permet de représenter d'une façon globale tous les acteurs de l'application. Dans ce cas : le Visiteur, le Client, le Staff et l'Administrateur.

Ce type de diagramme permet de voir clairement quelles actions chacun peut effectuer. C'est **une vue globale** et simplifiée : on ne parle pas encore de base de données ni d'appels API, on parle uniquement de ce que l'utilisateur peut faire depuis son point de vue. Ce diagramme aide à ne rien oublier lors du développement, à bien séparer les droits et responsabilités de chaque type d'utilisateur, et à avoir une référence visuelle à consulter quand nous doutons de ce qu'une fonctionnalité devait réaliser.

J'ai réalisé ce diagramme avec l'outil en ligne draw.io, qui propose des formes **UML (Unified Modeling Language, un langage de modélisation graphique)** prêtes à l'emploi et permet d'obtenir un résultat lisible facilement.

Le deuxième outil est le diagramme de séquence. **Contrairement au diagramme de cas d'utilisation qui répond à la question "qui peut faire quoi ?", le diagramme de séquence répond à la question "comment ça se passe, et dans quel ordre ?"**. Pour chaque acteur, j'ai modélisé son flux principal : quels messages sont échangés entre le navigateur, le frontend React, l'API Symfony, la base de données MySQL ou MongoDB, et le service d'envoi d'emails Brevo. Ces diagrammes permettent de comprendre la logique complète d'une fonctionnalité. Par exemple, tout ce qui se passe entre le moment où un client clique sur "Confirmer ma commande" et le moment où il reçoit son email de confirmation implique plusieurs systèmes qui doivent communiquer dans le bon ordre. J'ai réalisé ces diagrammes avec [Mermaid Live Editor](https://mermaid.live), un outil qui génère automatiquement le visuel à partir d'un code textuel simple.

Ces deux types de diagrammes sont complémentaires : le premier donne la vue d'ensemble fonctionnelle, le second explique les interactions techniques. Ensemble, ils constituent une documentation de conception qui aide à comprendre l'application.

Diagramme de cas d'utilisation

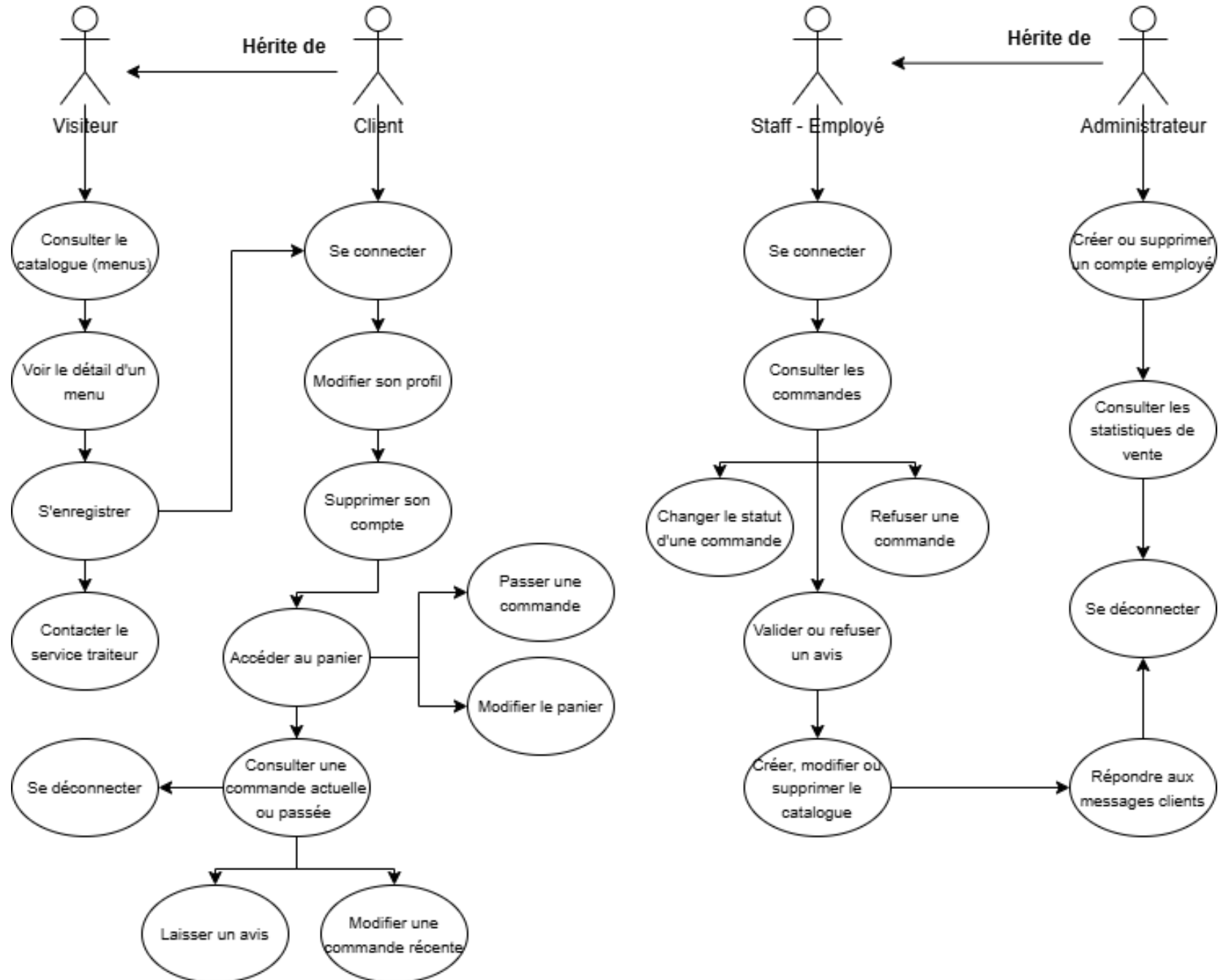


Diagramme de séquence pour un visiteur

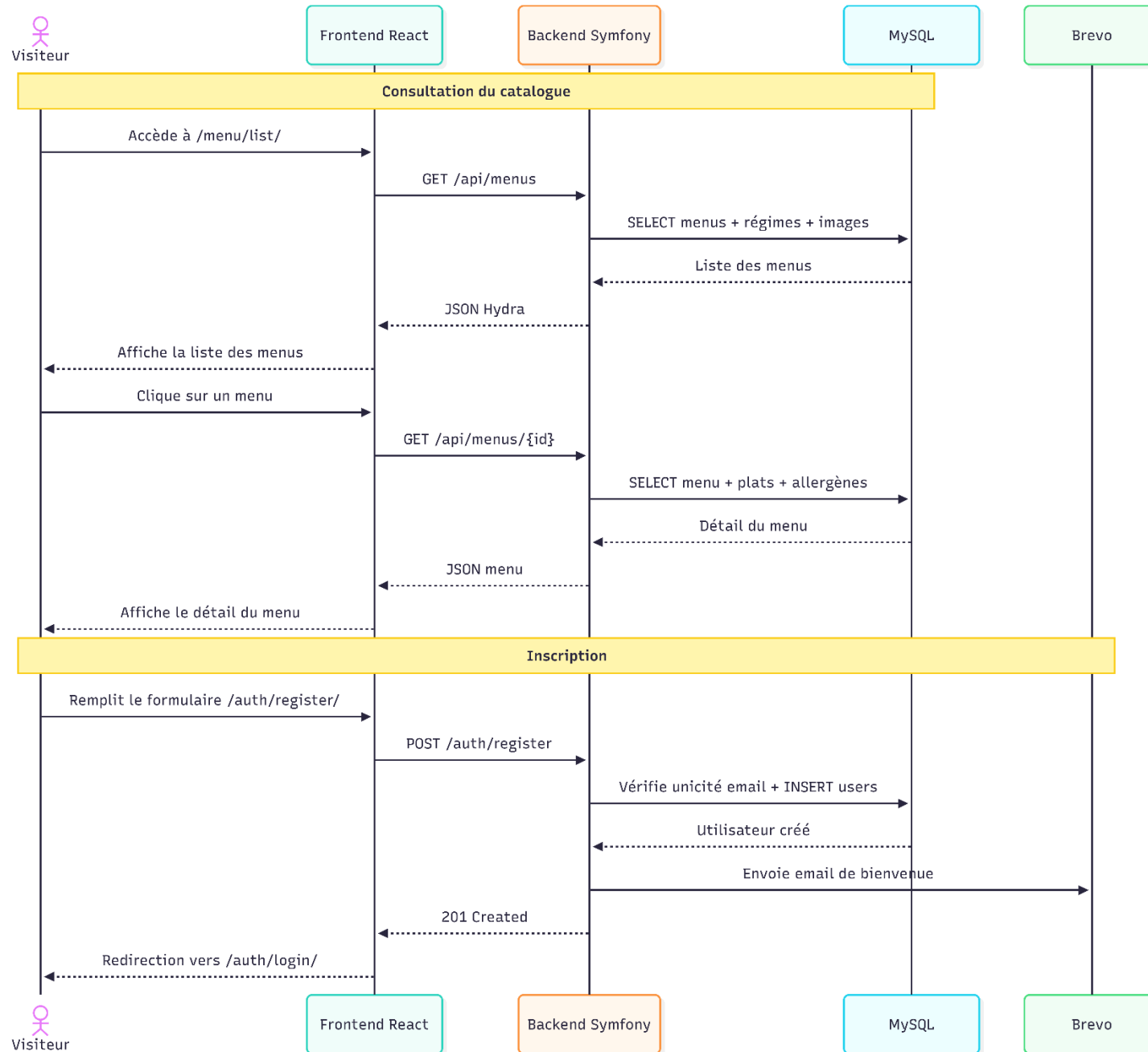


Diagramme de séquence pour un client connecté

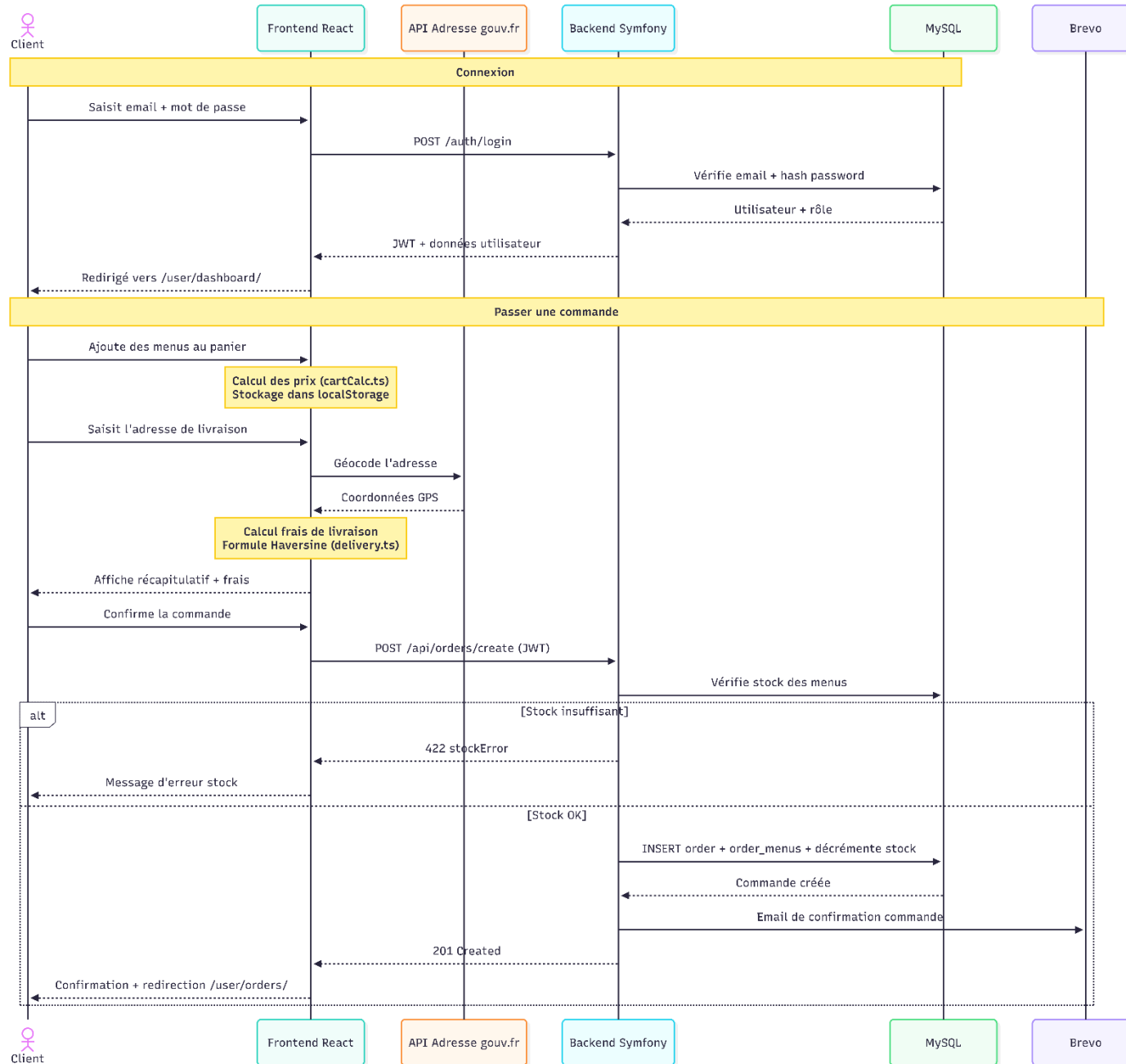


Diagramme de séquence pour un employé du staff

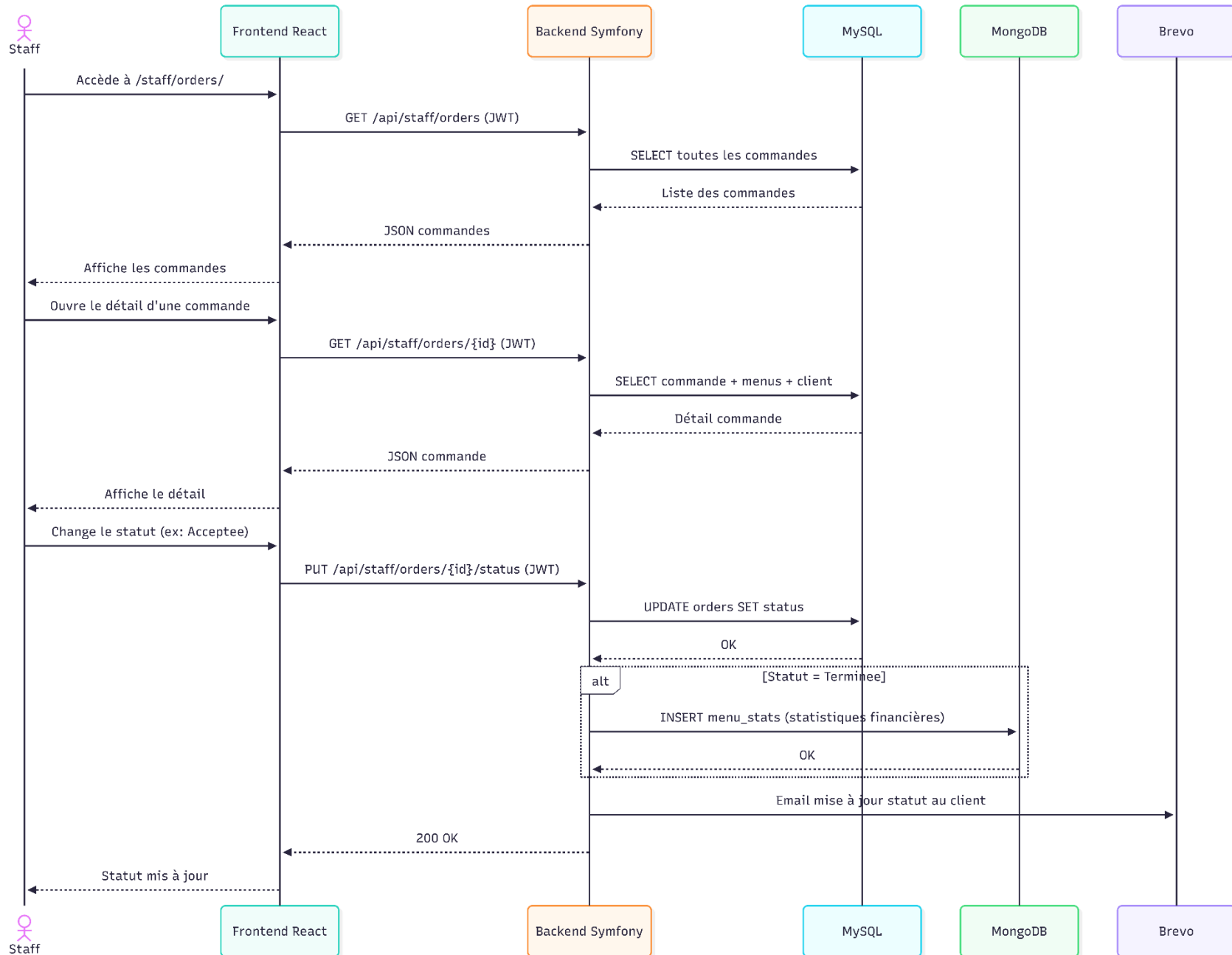


Diagramme de séquence pour un administrateur

